

오만가지 소스 - 풀이

작성자: 조승현

N 개의 원소를 6개의 크기가 비슷한 집합 $S_1, S_2, \dots, S_6$ 으로 나누어서, 임의의 3개를 합집합해도 크기가 $\lceil N/2 + 1 \rceil$ 이하가 되게 할 수 있다.

이제, 모든 S_i 들, $S_i \cup S_j$ 들, $S_i \cup S_j \cup S_k$ 들에 대해 질의하여 답을 얻었다고 하자. ($1 \leq i, j, k \leq 6$) 쿼리의 총 횟수는 $\binom{6}{1} + \binom{6}{2} + \binom{6}{3} = 41$ 회이다.

한편, 이 쿼리들을 통해

X 에서 S_i 에 포함되는 원소의 개수(= $\text{query}(S_i)$),

S_i 및 S_j 와만 교집합이 있는 원소의 개수(= $\text{query}(S_i \cup S_j) - \text{query}(S_i) - \text{query}(S_j)$)

S_i, S_j, S_k 에 걸쳐있는 원소의 개수(= $\text{query}(S_i \cup S_j \cup S_k) - \text{query}(S_i \cup S_j) - \text{query}(S_i \cup S_k) - \text{query}(S_j \cup S_k) + \text{query}(S_i) + \text{query}(S_j) + \text{query}(S_k)$)

를 알 수 있다.

X 의 각 원소는 이 중 정확히 하나에 포함되므로, 모두 합해주면 답이 된다.

정렬하기 - 풀이

작성자: 박영우

물건을 정점으로, 실을 간선으로 생각하면 한 질문(쿼리)의 반환값은 트리에서의 독립 집합(independent set) 중 가중치가 가장 큰 것이다.

다음과 같은 방법으로 어떤 두 정점 a, b 에 대해 $A[a] \leq A[b]$ 임을 확인할 수 있다.

세 점 u, v, x 를 고르고 나머지 정점들의 집합을 S 라 하자. u, v, x 를 순서대로 직선으로 연결하고 S 에 속한 정점을 전부 x 에 연결한 후 쿼리를 호출한다.

쿼리의 반환값에서 x 가 선택된 경우와 그렇지 않은 경우로 나눌 수 있다.

- x 가 선택되지 않은 경우: $A[u], A[v]$ 중 하나가 0이 아니라면 $A[u], A[v]$ 중 작지 않은 값이 선택되었을 것이므로 $A[u] \leq A[v]$ 혹은 $A[v] \leq A[u]$ 임을 확인할 수 있다. u, v 중 아무 정점도 선택되지 않은 경우 $A[u] = A[v] = 0$ 임을 알 수 있다.
- x 가 선택된 경우: S 에 있는 정점들의 A 값의 합보다 $A[x]$ 가 더 작다면 x 를 선택하지 않고 S 에 있는 정점을 선택하는 것이 이득이므로 S 의 임의의 정점 w 에 대해 $A[x] \geq A[w]$ 이다.

비슷하게, $A[u] \leq A[v]$ 임을 아는 쌍이 있다면 u, v 를 포함하지 않고 서로 겹치지 않는 순서쌍 $(x_1, y_1), \dots, (x_k, y_k)$ 가 있을 때 각각의 i 에 대해 $A[x_i] \leq A[y_i]$ 이거나 $A[x_i] \geq A[y_i]$ 중 하나를 알 수 있다. x_i, y_i, u 를 순서대로 연결하고, u 와 v 를 연결하고, 위 과정에서 어느 정점과도 연결되지 않은 모든 정점을 u 와 연결한 후 쿼리를 호출하면 된다.

u 가 선택된 경우와 그렇지 않은 경우로 나눌 수 있다.

- u 가 선택되지 않은 경우, 앞선 논리와 비슷하게 x_i 와 y_i 중 하나가 선택되었다면 $A[x_i] \leq A[y_i], A[x_i] \geq A[y_i]$ 중 하나를 알 수 있고, 둘 다 선택되지 않았다면 $A[x_i] = A[y_i] = 0$ 이다.
- u 가 선택된 경우 $A[u] \leq A[v]$ 이므로 반환된 집합에서 u 를 제외시키고 v 를 추가한 집합도 최대 독립 집합이다. 따라서 이 집합에서 이전 논리를 적용하면 $A[x_i] \leq A[y_i], A[x_i] \geq A[y_i]$ 중 하나를 알 수 있다.

위의 방법을 이용하여 처음 $A[u] \leq A[v]$ 임을 알아낸 u, v 를 제외한 $N - 2$ 개의 원소를 분할 정복을 통해 수열을 정렬할 수 있다. 수열을 두 부분으로 분할하면 각각을 정렬하는 것은 병렬적으로 진행할 수 있다. 길이가 같은 정렬된 두 수열을 하나의 정렬된 수열로 병합하는 것은 다음과 같이 할 수 있다.

- 길이 $2N$ 이고 앞쪽 절반과 뒤쪽 절반이 각각 정렬되어있는 배열 A 를 다음과 같이 정렬할 수 있다.
 N 은 짝수이며 A 의 모든 원소가 서로 다르다고 가정할 수 있다.
 우선 A 의 홀수번째 인덱스, 짝수번째 인덱스를 따로 모아 이 인덱스들끼리 정렬한다.
 이렇게 하여 만들어진 배열을 A' , A 를 정렬하면 얻을 수 있는 배열을 A'' 이라 하자.
 A' 의 홀수 번째 인덱스 $A'[2k + 1]$ 에 대해 생각하자. 일반성을 잃지 않고 이 인덱스가 A 의 앞쪽 절반에 위치했다 하자.

A 의 홀수번째 원소들 중 $A'[2k+1]$ 보다 작은 것은 총 k 개 존재하는데, 배열 앞쪽 절반에서 a 개, 뒤쪽 절반에서 b 개 존재한다 하자.

그러면 배열 앞쪽 절반에서는 $A'[2k+1]$ 보다 작은 원소가 $2a+1$ 개($A'[2k+1] = A[2a+1]$ 이므로), 뒤쪽 절반에서는 $2b+1, 2b+2$ 개 중 하나이다. ($A[2b+1] < A'[2k+1] < A[2b+3]$)

따라서 $A'[2k+1]$ 은 $A''[2k+1], A''[2k+2]$ 중 하나이다.

마찬가지로 짝수 번째 인덱스 $A'[2k]$ 는 $A''[2k]$ 나 $A''[2k-1]$ 중 하나이다.

따라서 A' 에서 모든 $2k+1, 2k+2$ 쌍에 대해 각각 정렬해주면 A'' 을 얻을 수 있다.

크기 2^k 의 두 수열을 합치는 것을 $k+1$ 회의 연산으로 할 수 있고, 따라서 2^k 크기의 수열을 정렬하는 것은 $k(k+1)/2$ 회에 할 수 있다. 이렇게 하면 $N-2$ 개의 값을 정렬하는 것을 55회에 할 수 있다.

또한, 길이가 $N-2$ 인 정렬된 배열에 처음에 구한 대소 관계를 아는 2개의 수 u, v 를 끼워넣는 과정은 다음과 같이 할 수 있다.

N 이 10 이하인 경우 u, v 를 제외하고 대소관계를 아는 수 세 개가 있어 ($5 \leq N$), u 와 다른 수를 비교하는 것이 가능하므로 u, v 와 나머지 $N-2$ 개의 쌍의 대소관계를 전부 비교해 보는 방법으로 풀 수 있다.

N 이 큰 경우에는 분할 정복을 사용한다. 배열의 가운데에 있는 수를 x, y 라 할 때, u, x 와 y, v 를 동시에 비교한다. $A[u] \leq A[v], A[x] \leq A[y]$ 이므로 결과는 다음 중 하나이다.

- $A[u] \leq A[x], A[y] \leq A[v]$: 이 경우 u 가 앞쪽 절반에 들어가고, v 가 뒤쪽 절반에 들어가므로 이분 탐색을 통해 찾을 수 있다. 이때 두 이분 탐색을 병렬적으로 시행할 수 있다.
- $A[x] \leq A[u], A[v] \leq A[y]$: $A[x] \leq A[u] \leq A[v] \leq A[y]$ 임을 알 수 있으므로 x, y 사이에 u, v 가 들어가게 된다.
- $A[u] \leq A[x], A[v] \leq A[y]$: 두 수 모두 앞쪽 절반에 위치한다.
- $A[x] \leq A[u], A[y] \leq A[v]$: 두 수 모두 뒤쪽 절반에 위치한다.

따라서 전체 $O(\lg N)$ 쿼리에 해결할 수 있고, 최악의 상황에서도 12회 이하의 쿼리를 사용함을 보일 수 있다.

따라서 사용하는 총 쿼리의 횟수는 최대 $1 + 55 + 12 = 68$ 회로, 주어진 제한 조건을 충족한다.

날다람쥐 2 - 풀이

작성자: 박상훈

부분문제 1

다음과 같은 dp 를 정의하자.

$dp[i][j][k]$: 수평 위치 i 까지 왔을 때, 높이가 j 이고 점프 횟수가 k 일 때 최소 시간, 불가능하면 ∞

상태 수는 $O(N^3)$ 이고 전이는 $O(1)$ 이므로 시간복잡도 $O(N^3)$ 에 문제를 해결할 수 있다.

부분문제 2

$L[i] \leftarrow \max(L[i-1], L[i])$, $R[i] \leftarrow \min(R[i], R[i+1])$ 을 해도 문제는 동일하다. 따라서, 앞으로 L 과 R 이 단조증가한다고 가정한다.

조건을 만족하는 경로가 존재하는지 판별하면 충분하다. R 이 단조증가하므로 1만큼 올라가는 것이 가능할 때마다 올라가는 그리디 알고리즘을 구현하면 부분문제를 해결할 수 있다.

부분문제 3

편의상 $L[N] = R[N] = H$ 라고 하자.

높이 h ($1 \leq h \leq H$)에 대해, $s_h = \min\{i | R[i] \geq h\}$, $e_h = \min\{i | L[i] \geq h\}$ 라고 하자. 높이 $h-1$ 에서 h 로 올라가는 것이 가능한 수평 위치는 구간 $[s_h, e_h]$ 와 동일하다. L 과 R 이 단조증가하므로 s 와 e 도 단조증가한다.

수평 위치 i ($0 \leq i \leq N-1$)가 구간 $[s_h, e_h]$ 에 속할 때만 간선으로 연결하여 만든 이분그래프를 생각하자. s 와 e 가 단조증가하므로 임의의 크기 H 의 매칭에 대해, 매칭에 속한 수평 위치에서 높이를 1씩 증가시키는 해가 존재한다. 따라서, 최소 비용 최대 매칭을 구하는 문제로 생각할 수 있다.

다음 그리디 알고리즘으로 얻은 S 가 최적해임은 잘 알려져 있다(MCMF):

1. $S = \emptyset$ 으로 시작한다.
2. $A[i]$ 가 증가하는 순으로 보면서, $S \cup \{i\}$ 를 덮는 매칭이 존재한다면 $S \leftarrow S \cup \{i\}$ 을 반복한다.

주어진 집합 S 와 고정된 양의 실수 $\epsilon < \frac{1}{4H}$ 에 대해, 다음 방식으로 구성한 문자열 $str(S)$ 을 생각하자:

1. 수평 위치 $i \in S$ 에 * 을 적는다.
2. 수평 위치 $s_h - h\epsilon$ 에 (를 적는다.
3. 수평 위치 $e_h + h\epsilon$ 에) 를 적는다.
4. 수평 위치가 증가하는 순으로 문자를 이어붙인다.

홀의 정리에 의해, S 를 덮는 매칭이 존재할 필요충분조건은 $str(S)$ 의 부분수열(연속하지 않아도 됨) 중 $((...))^{**...}^{**}((...(($ 꼴의 부분수열의 최대 길이가 H 이하이다.

따라서, 앞서 말한 그리디 알고리즘을 세그먼트 트리를 사용하여 구현할 수 있다. 시간복잡도는 $O(N \log N)$ 이다.

부분문제 4

편의상 $A[i] \leftarrow A[i] + 2^{-i-100}$ 를 했다고 생각한다. 이렇게 하면 모든 서로 다른 집합에 대해서 비용의 합이 다르다. $T[i] \neq -1$ 인 i 에 대해, $T[i]$ 에 대응되는 집합을 $S(i)$ 라고 하자. 또한, 부분문제 3에서와 정의한 대로 해당 S 와 구간들 간에 매칭이 존재하는 점프 위치의 부분집합 S 를 좋은 집합이라고 정의한다.

다음 사실이 성립한다:

1. $T[i] \neq -1$ 을 만족하는 i 는 구간 $[l, r]$ 로 표현 가능하다.
2. $l \leq i \leq r - 1$ 인 임의의 i 에 대해, $S(i)$ 에서 $B[x] = 0$ 인 원소를 하나 제거하고 $B[x] = 1$ 인 원소를 하나 추가하여 $S(i + 1)$ 을 만들 수 있다.
3. $l \leq i \leq r$ 에서 $T[i]$ 는 볼록하다.

이 사실은 모두 다음 정리에 의해 따라온다:

- S_1, S_2 를, 좋은 집합이라고 하자. 임의의 $x \in S_1 \setminus S_2$ 에 대해서, 어떠한 위치 $y \in S_2 \setminus S_1$ 이 존재하여 $S_1 - \{x\} + \{y\}$ 와 $S_2 - \{y\} + \{x\}$ 가 모두 매칭 조건을 만족한다.

S_1 과 구간간의, S_2 와 구간간의 매칭을 그려보면, 임의의 $x \in S_1 \setminus S_2$ 에 대해서 x 와 매칭된 구간과 매칭된 S_2 의 원소는 $S_2 \setminus S_1$ 에 속함을 알 수 있다. 두 원소를 교환하는 것으로 다른 매칭을 해치지 않고 두 집합이 모두 매칭 조건을 만족함을 관찰할 수 있고, 증명이 종료된다. 매트ROID 이론에 익숙하다면, 좋은 집합이 Transversal Matroid의 Basis에 대응되며, 위 정리는 Strong Base Exchange Theorem에 따라 자명함을 알 수 있다.

이제 위 사실을 모두 증명할 수 있다.

- (2) $S(i) - S(i + 1)$ 에서 $B[x] = 0$ 인 원소를 찾자. 이에 따라 대응되는 $y \in S(i + 1) - S(i)$ 도 찾을 수 있다. 만약 $B[y] = 0$ 인 경우, $A[x] \neq A[y]$ 이기 때문에 둘 중 하나는 최적해가 아니다. $B[y] = 1$ 인 경우, $T[i] - A[x] + A[y] = T[i + 1]$ 이 아닐 경우 둘 중 하나는 최적해가 아니다.
- (1) 위 증명에 따라서 $S(l) - S(r)$ 에서 원소를 찾는 식으로 모든 $S(l + 1), S(l + 2), \dots, S(r - 1)$ 을 구성할 수 있다.
- (3) $T[i] - T[i - 1] > T[i + 1] - T[i]$ 인 i 가 존재한다고 하자. $S(i - 1) - \{x_1\} + \{y_1\} = S(i), S(i) - \{x_2\} + \{y_2\} = S(i + 1)$ 이라고 하자 ($B[x_1] = B[x_2] = 0, B[y_1] = B[y_2] = 1$). $S(i - 1) - \{x_2\} + \{y'\}, S(i + 1) - \{y'\} + \{x_2\}$ 가 위 정리에 따라 좋은 집합이라고 하자. $S(i)$ 의 최적성에 의해 $A[y'] - A[x_2] > A[y_1] - A[x_1]$ 이다. 또한 가정에 의해 $A[y_1] - A[x_1] > A[y_2] - A[x_2]$ 이다. 고로 $A[y'] > A[y_2]$ 이며, $S(i + 1) - \{y'\} + \{x_2\}$ 가 $S(i)$ 보다 더 낮은 비용을 가지게 되어 모순이다.

T 가 볼록하며, 가능한 기울기의 수가 $\max |A_i|$ 에 비례하니, $A'[i] = A[i] - B[i] \times \lambda$ 로 두고 부분문제 3을 풀면 T 에서 기울기가 변하는 점을 모두 구할 수 있다. (Aliens' trick, Lagrange optimization) 가능한 모든 기울기를 시도해보면 부분문제 4를 해결할 수 있다. 시간복잡도는 $O(\max |A_i| \times N \log N)$ 이다.

부분문제 6

$R = S(l) - S(r)$, $B = S(r) - S(l)$ 라고 하자. $S(l)$ 에서 시작해서 R 의 원소를 하나 지우고, B 의 원소를 하나 추가하는 것을 반복하여 $S(l+1), \dots, S(r)$ 을 만들 수 있다. $S(i)$ 에서 $S(i+1)$ 로 갈 때 제거한 원소를 r_i , 추가한 원소를 b_i 라고 하자. 집합 $D = \{(r_l, b_l), (r_{l+1}, b_{l+1}), \dots, (r_{r-1}, b_{r-1})\}$ 을 구하면 충분하다: 집합의 각 원소가 구체적으로 언제 지워졌는지를 알지 못한다 해도, 볼록성에 의해 정렬할 경우 쉽게 그 시점을 역산할 수 있다.

어떠한 좋은 집합 S 와 위치 $b \in S$ 에 대해서, $(S \cup \{b\}) - \{c\}$ 가 좋은 집합인 모든 $c \in S$ 를, b 가 이루는 서킷이라고 정의한다. 서킷은 스페닝 트리의 사이클과 유사한 개념이라고 생각하면 되며, 매트ROID 이론에서 사용하는 용어를 그대로 차용하였다.

다음 그리디 알고리즘은 D 를 올바르게 구함을 증명할 수 있다:

1. $D = \emptyset$, $S = S(l)$ 에서 시작한다.
2. B 의 원소를 비용 오름차순으로 보면서 반복한다. $b \in B$ 에 대해, b 가 이루는 서킷에 속한 가장 비용이 큰 $r \in R$ 을 선택한다. $D \leftarrow D \cup \{(r, b)\}$, $S \leftarrow (S \cup \{b\}) - \{r\}$ 를 한다.

$S \cup \{b\}$ 의 서킷에 속하는 원소는, $str(S \cup \{b\})$ 의 모든 $((\dots))^{**}\dots^{**}((\dots((\text{폴 길이 } H+1$ 부분수열에서 $*$ 에 속해야 한다. 이는 이와 같은 부분수열 중 $*$ 의 개수가 최소인 부분수열에 속함과 동치임을 보일 수 있다. 따라서, 서킷에 속한 가장 비용이 큰 r 은 세그먼트 트리로 구할 수 있다. 고로 위 그리디 알고리즘을 $O(N \log N)$ 에 구현할 수 있다.

관측탑 - 풀이

작성자: 이종영

부분문제 1

$i < j, H[i] < H[j]$ 인 최소 j 를 $P[i]$ 로 정의하자. (편의상 $H[N] = \infty$ 로 정의한다.) i 번 관측탑에서 관측 가능한 관측탑들은 $P[i], P[P[i]], \dots$ 이다. 트리 T 를 $P[i] \rightarrow i$ 간선들로 만들어진 루트가 N 인 트리로 정의하자. 이제 i 번 관측탑에서 관측 가능한 관측탑들은 i 부터 루트까지의 경로 상의 관측탑들이다.

관측탑들을 길이 $N^{0.5}$ 의 구간들로 나누면, 지각 변동 이벤트에서 상대적인 높이가 변화하는 구간들은 최대 2개이다. 또한 $i \rightarrow P[i]$ 에서 소속 구간이 바뀌는 횟수는 $O(N^{0.5})$ 번이다. 이를 이용해 같은 구간 내의 $i \rightarrow P[i]$ 만 관리한다면, 다음 구간으로 넘어가는 경우는 해당 구간에서 prefix maximum들만 고려해도 되기 때문에 이분 탐색을 통해 다음 지점을 구할 수 있다.

부분문제 2

부분문제 1과 유사하게 관측탑들을 길이 $N^{0.5}$ 의 구간들로 나눈다. 각 i 에 대해, 각 구간에서 관측 가능한 관측탑의 수를 미리 구한다. 이제 측정 이벤트에서 $[L, R]$ 에 완전히 포함된 구간들은 해당 값들을 이용하면 구할 수 있다. 나머지 부분은 부분문제 1과 유사하게 처리할 수 있다.

부분문제 3

부분문제 1의 풀이에서 이미 관측되는 관측탑들은 구할 수 있다. 이를 쿼리 처리에도 유사하게 처리할 경우 관측 점수 또한 관리가 가능하다.

부분문제 4

모든 지각 변동 이벤트에서 $L = R$ 임을 이용해 오프라인 동적 연결성 알고리즘과 유사하게, 모든 업데이트를 높이가 지정되지 않았던 특정 관측탑의 높이를 지정하는 것으로 생각한다. 이제 높이들을 관리하는 세그먼트 트리 내에서 이분 탐색을 통해 해당 관측탑이 가리게 되는 관측탑의 구간을 구할 수 있다. 0번 관측탑에서 관측 가능한 관측탑들은 다른 관측탑이 가리지 않는 관측탑들이므로, 세그먼트 트리 등을 통해 각 관측탑이 가리는 구간을 관리하면 관측 이벤트 또한 처리 가능하다. 총 시간 복잡도는 $O(N \log N + Q \log^2 N)$ 이다.

부분문제 5

$j < i, H[i] \leq H[j]$ 인 최대 j 를 $P[i]$ 로 정의하자. (편의상 $H[-1] = 0$ 로 정의한다.) i 번 관측탑에서 j 번 관측탑이 관측 가능할 필요충분조건은 $P[j] < i < j$ 이다.

모든 지각 변동 이벤트에서 $L = R, V \geq 0$ 이므로 $P[L]$ 을 제외한 P 값들은 L 로 증가하는 것만 가능하다. 결국 구간에 $P[i] = \max(P[i], L)$ 업데이트를 적용하는 것이므로, 세그먼트 트리 비츠를 적용하면 P 값들을 관리할

수 있다. 업데이트를 적용할 구간 및 $P[L]$ 의 값은 H 값들의 최댓값 세그먼트 트리에서 이분 탐색을 통해 구할 수 있다.

이제 관측 이벤트와 측정 이벤트를 처리하는 방법을 알아본다. $\{j : P[j] < i < j\} = \{j : P[j] < i\} \setminus \{j : j \leq i\}$ 임을 이용하자. $[1, i)$ 범위의 P 값의 관측 점수에 1을 더하고, $[1, i]$ 범위의 인덱스의 관측 점수에 -1 을 더하는 업데이트를 하면 된다. 이제 측정 이벤트는 $P[L]$ 의 관측 점수와 L 의 관측 점수의 합이 된다. P 값이 변하는 경우, P 값의 관측 점수의 변화가 있으므로 인덱스의 관측 점수에 해당 변화만큼 빼 주어야 한다. 이는 세그먼트 트리 비츠 과정에서 자연스럽게 적용할 수 있다. 시간 복잡도 $O((N + Q) \log^2 N)$ 등에 해결할 수 있다.

부분문제 6

다음과 같이 세그먼트 트리를 구성하자. 각 구간 $[L, R]$ 을 나타내는 노드 u 에 대해, 관리하는 구간 $[L, R]$ 을 $range(u)$, 왼쪽 자식을 $left(u)$, 오른쪽 자식을 $right(u)$ 라고 하자. 관리하는 값들은 구간 내 높이 최댓값 $max(u)$, 높이 0에서 관측을 시작해 $range(u)$ 를 지나갔을 때 지나가는 관측값의 수 $jumps(u)$, $max(left(u))$ 에서 관측을 시작해 $range(right(u))$ 를 지나갔을 때 지나가는 관측값의 수 $rjumps(u)$.

$jumps$ 값들과 $rjumps$ 값들은 세그먼트 트리 내에서 이분 탐색을 이용하면 각 노드마다 $O(\log N)$ 에 구할 수 있다. 구체적으로, 현재 높이 h 에 대해, $h < max(left(u))$ 인 경우 $left(u)$ 에서 h 로 관측을 진행한 후, $right(u)$ 에서 $max(left(u))$ 로 관측을 진행하기에 $rjumps(u)$ 가 더해진다. $h \geq max(left(u))$ 인 경우 $right(u)$ 에서 h 로 관측을 진행하면 된다.

지각 변동 이벤트는 lazy propagation을 사용하면 처리할 수 있다. 관측 이벤트는 일반적인 세그먼트 트리에서의 쿼리처럼 진행하되, 노드들 간에는 $jumps$ 값들을 구했던 과정을 비슷하게 반복하면 처리할 수 있다. 이제 $jumps$ 와 $rjumps$ 값들에 관여하는 노드들에만 전파하는 lazy propagation을 통해 관측 가능한 관측값들에만 업데이트를 처리할 수 있다. 자식들이 아닌 $O(\log N)$ 개의 자손들에 값을 전달해줘야 함에 유의하자. 이제 전체 문제를 $O(N \log N + Q \log^2 N)$ 에 해결할 수 있다.